User Manual of ValidationUtility **2**
Introduction **2**
Problems It Solves **2**
System Requirements 2
Installation Guide **3**
1. For Windows 3
2. For Mac OS 7
Getting Started 9
Rules and Validation Logic **15**
Error Handling 15
User Actions and Controls 16
FAQs **16**
Note 16

User Manual of ValidationUtility

This document provides comprehensive instructions on how to use ValidationUtility to govern and validate your API (OpenAPI) and Event (AsyncAPI) specifications.

Introduction

ValidationUtility is a desktop-based governance tool designed to automate the validation of API and Event specifications. It ensures that your YAML files strictly adhere to predefined naming conventions, data types, and formatting standards.

The primary purpose of ValidationUtility is to maintain a high level of consistency across all service definitions in an ecosystem. It reduces manual review time by automatically detecting violations and suggesting standardized fixes.

Problems It Solves

- **Inconsistent Naming:** Mix of `snake_case` and `PascalCase` across different specs.
- **Missing Documentation:** Fields without descriptions or examples.
- **Incorrect Data Types:** Dates not following ISO 8601 or booleans with improper naming.
- **Poor Formatting:** Inconsistent punctuation and sentence casing in descriptions.

System Requirements

OS Requirements

- Windows: Windows 10 or 11 (64-bit recommended).
- macOS: macOS 11 (Big Sur) or later.

Hardware Basics

- **RAM:** 4GB Minimum (8GB recommended for large YAML files).
- **Disk Space:** ~200MB for the application and its dependencies.

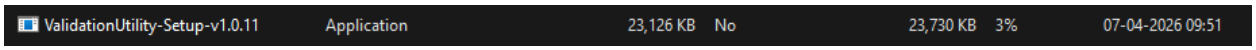
Installation Guide

1. For Windows

ValidationUtility is typically distributed as a standalone Windows executable (`ValidationUtility.exe`) or an installer.

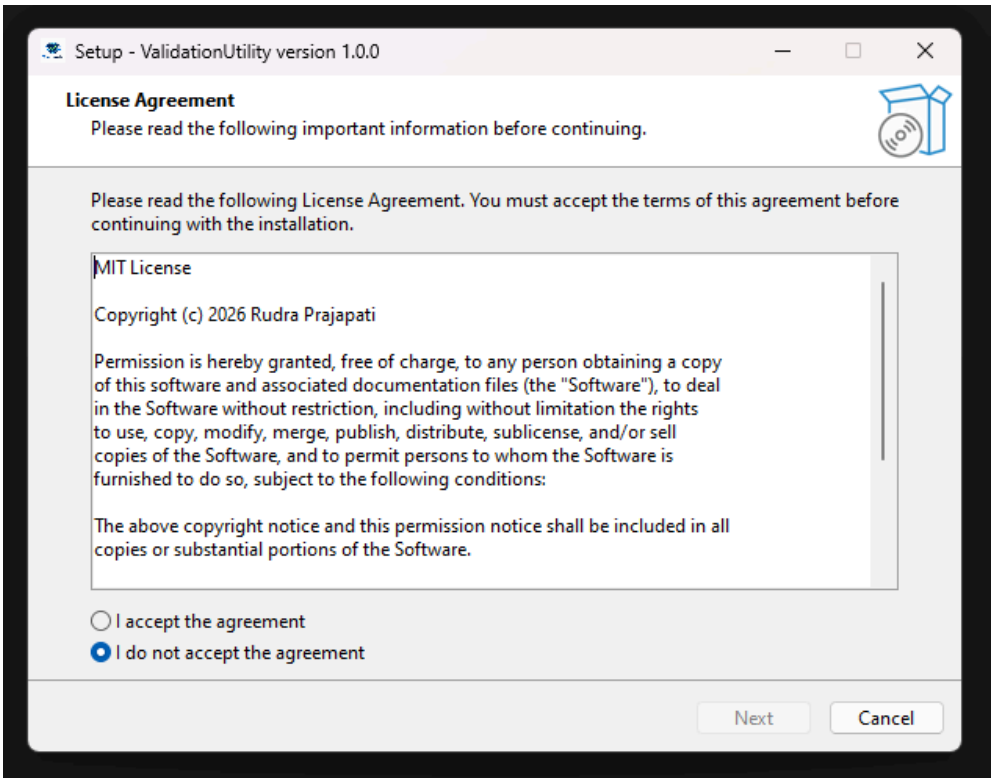
1. **Download**

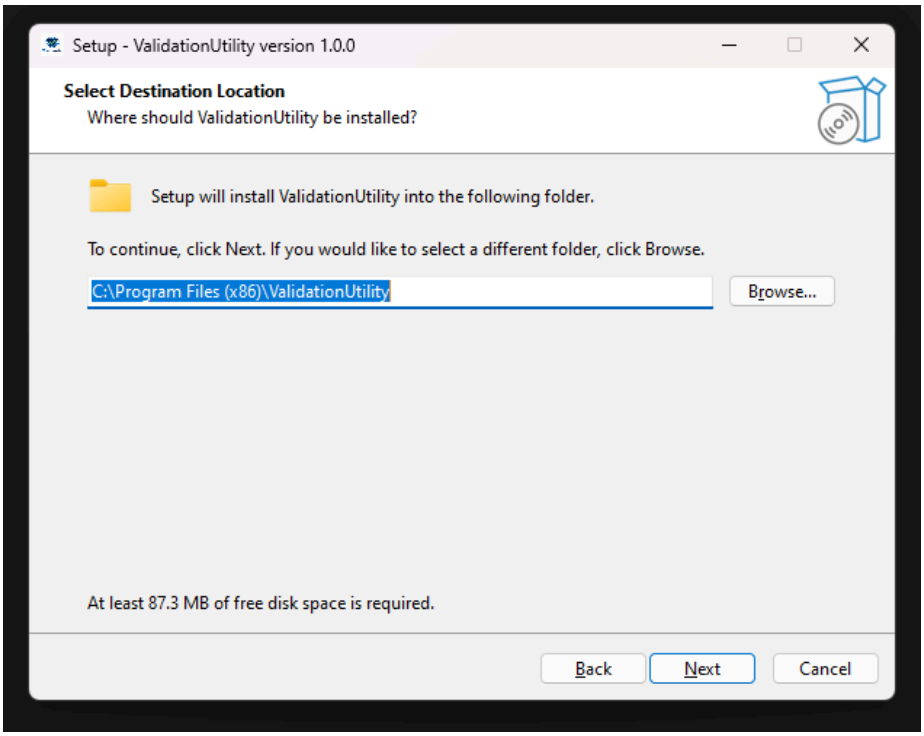
   [Download](#) the installer package of ValidationUtility.

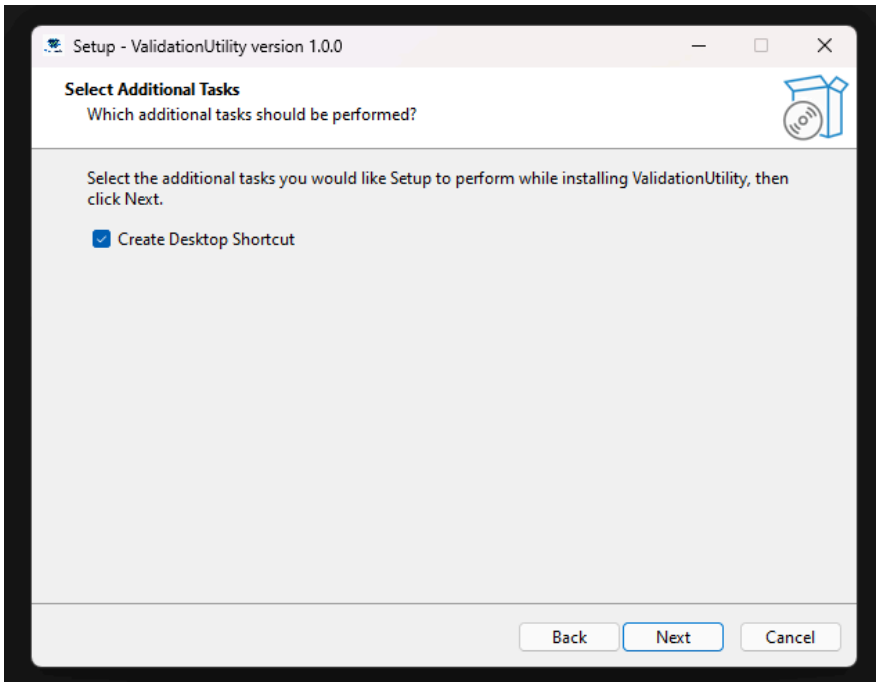


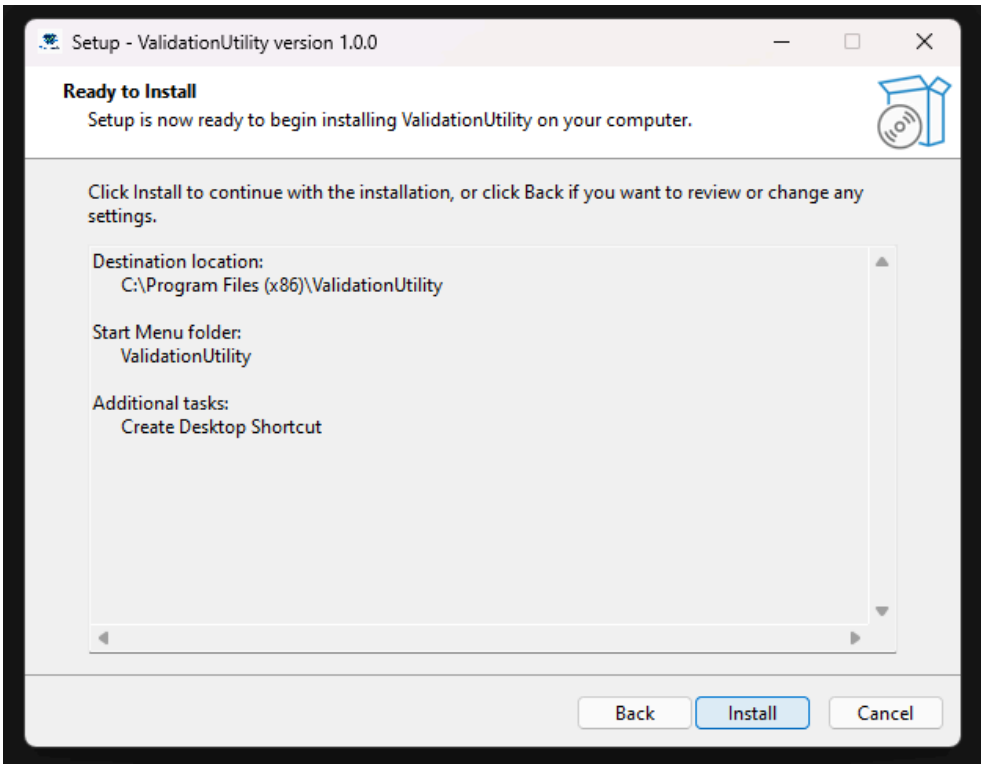
2. **Run Installer**

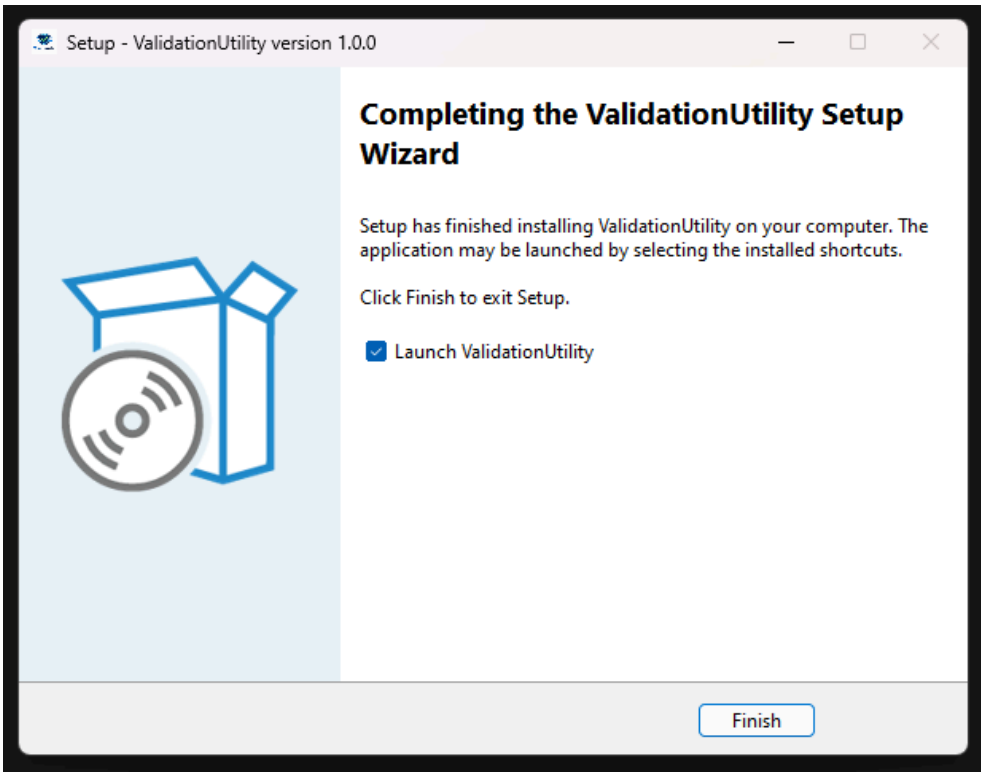
   Double-click on it and follow the on-screen prompts to install it in your 'Program Files' directory.



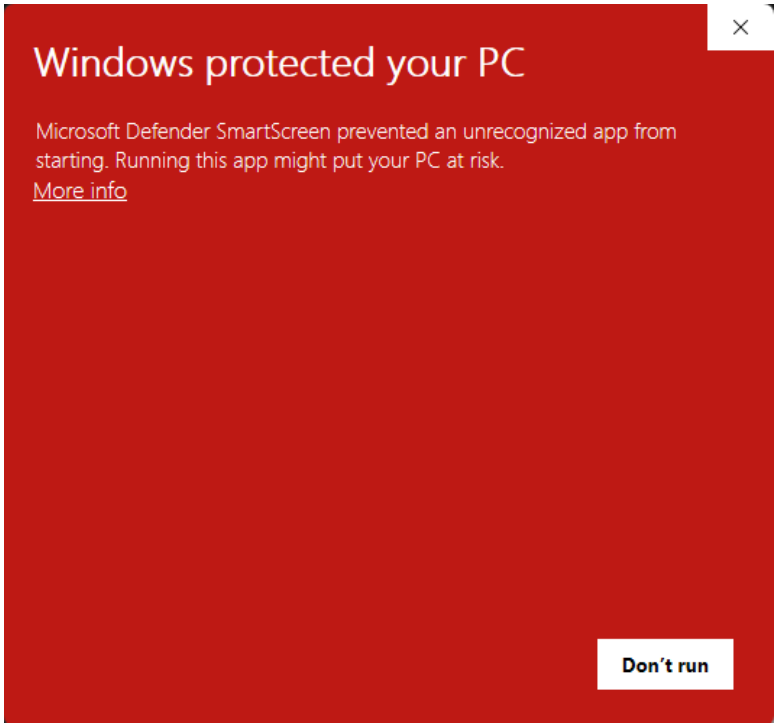

Setup - ValidationUtility version 1.0.0
Select Destination Location
Where should ValidationUtility be installed?
Setup will install ValidationUtility into the following folder.
To continue, click Next. If you would like to select a different folder, click Browse.
C:\Program Files (x86)\ValidationUtility
Browse...
At least 87.3 MB of free disk space is required.
Back
Next
Cancel

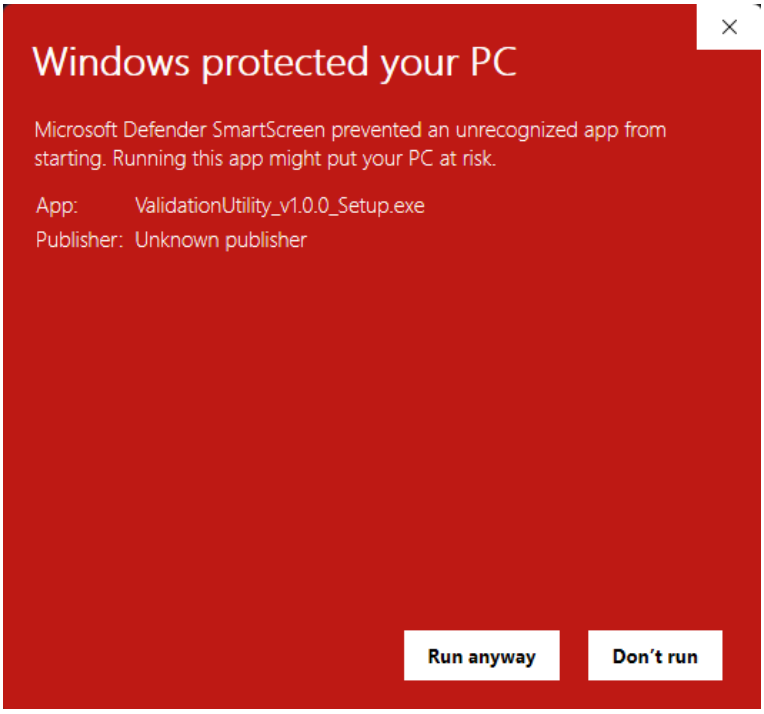

Setup - ValidationUtility version 1.0.0
Select Additional Tasks
Which additional tasks should be performed?
Select the additional tasks you would like Setup to perform while installing ValidationUtility, then click Next.
Create Desktop Shortcut
Back
Next
Cancel


Setup - ValidationUtility version 1.0.0
Ready to Install
Setup is now ready to begin installing ValidationUtility on your computer.
Click Install to continue with the installation, or click Back if you want to review or change any settings.
Destination location:
C:\Program Files (x86)\ValidationUtility
Start Menu folder:
ValidationUtility
Additional tasks:
Create Desktop Shortcut
Back
Install
Cancel


Setup - ValidationUtility version 1.0.0
Completing the ValidationUtility Setup Wizard
Setup has finished installing ValidationUtility on your computer. The application may be launched by selecting the installed shortcuts.
Click Finish to exit Setup.
Launch ValidationUtility
Finish

Important:
You may see a Windows security alert while installing the application. This happens because the app is not yet recognized by Microsoft. Simply click **"More info" → "Run anyway"** to proceed safely with the installation.





2. For Mac OS

This guide will walk you through setting up the ValidationUtility on your own Mac. By following these steps to build the application locally, you will avoid standard macOS security warnings.

1. **Download**

   Download the application for Mac OS.

2. **Extract**

   Extract the downloaded zip file, then double—click the ValidationUtility.app application.

3. **Important: If the application doesn't open**

   Because the app is an internal tool, macOS may show a "Malware" or "Unidentified Developer" warning. If this happens, follow these quick steps:

   A. Open Terminal

      Press "Cmd + Space" on your keyboard, type "Terminal", and press Enter.

   B. Run the Trust command

      Copy and paste the text below into the Terminal, but **Do not press Enter yet**:

      ```
      sudo xattr -rd com.apple.quarantine
      ```

      (Make sure there is a space after the word `quarantine`)

   C. Link the application

   1. Drag and drop the ValidationUtility.app icon directly into the Terminal window.

   2. The Terminal will fill in the path automatically. It should look like this:

      ```
      sudo xattr -rd com.apple.quarantine /path/to/ValidationUtility.app
      ```

3. Press Enter.
4. Type your Mac Password when prompted and press Enter.

   (Note: No characters will appear as you type).

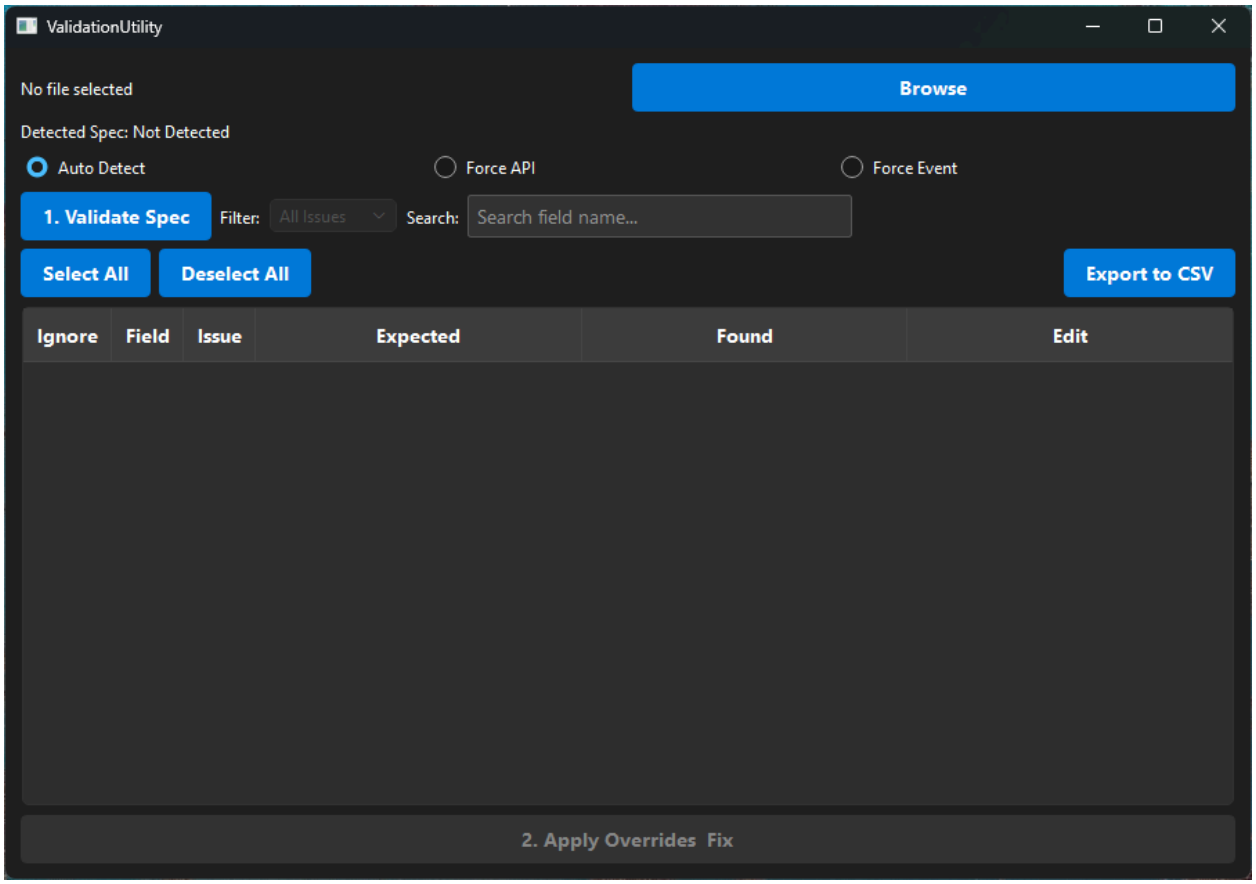
You can now close the Terminal and open the ValidationUtility app normally.

Getting Started

Follow this step-by-step workflow to validate your first specification:

1. **Launch Application**

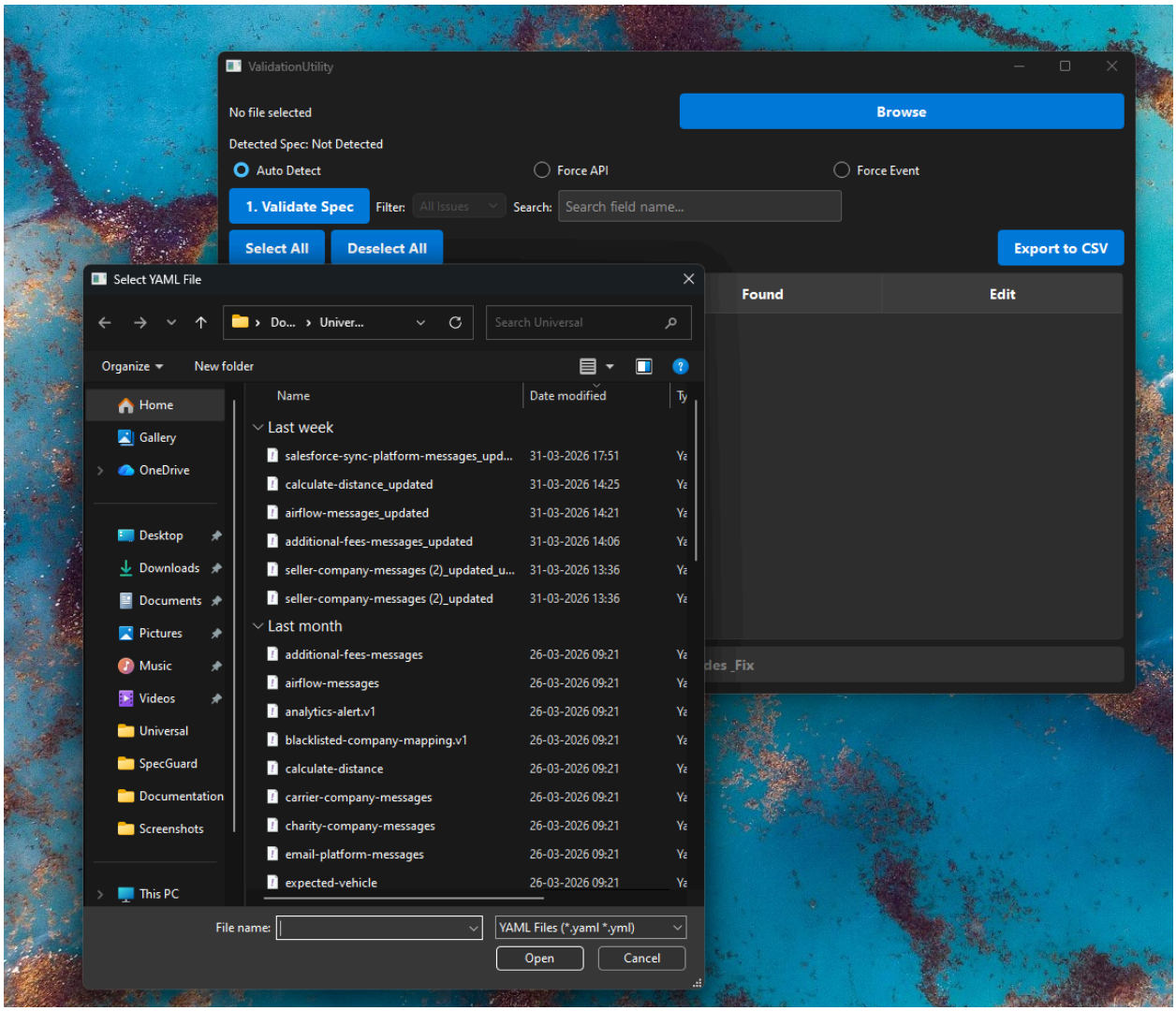
   Open 'ValidationUtility.exe'.



2. Upload YAML File

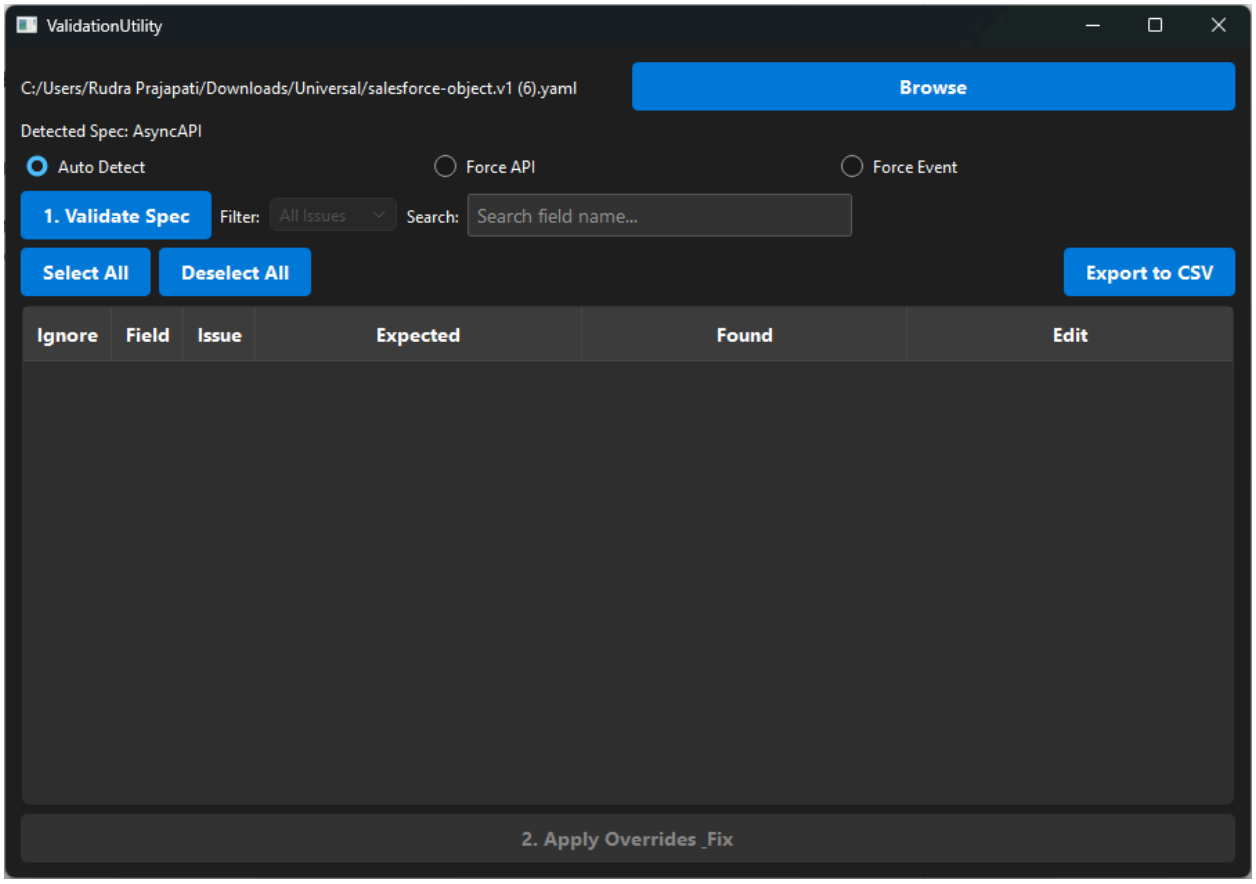
Click the Browse button and select the YAML file you wish to validate. ValidationUtility supports:

- Pure OpenAPI/AsyncAPI files.
- Backstage-wrapped API definitions.



3. Run Validation

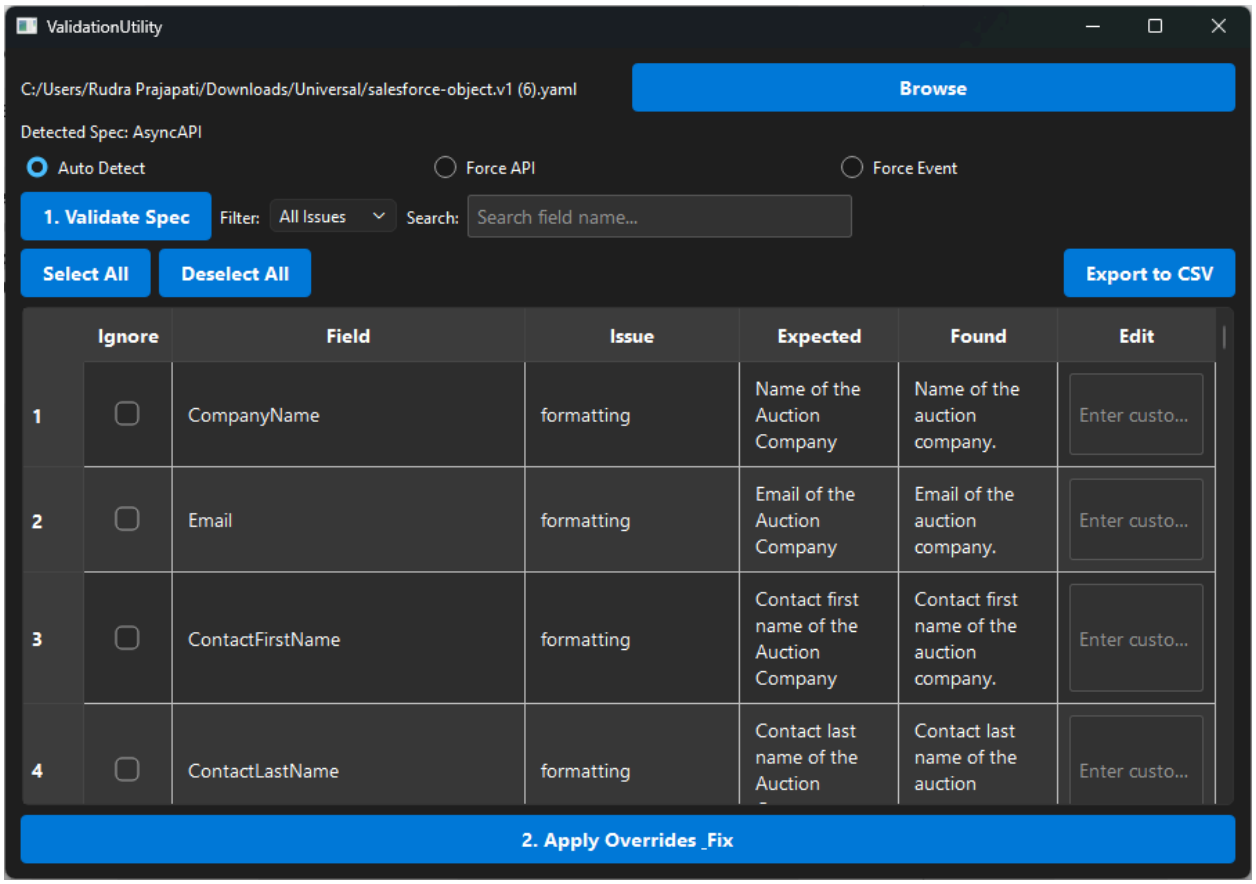
Click '1. Validate Spec'. ValidationUtility will automatically detect if the file is an OpenAPI or AsyncAPI spec and run the appropriate rules.



4. View Violations

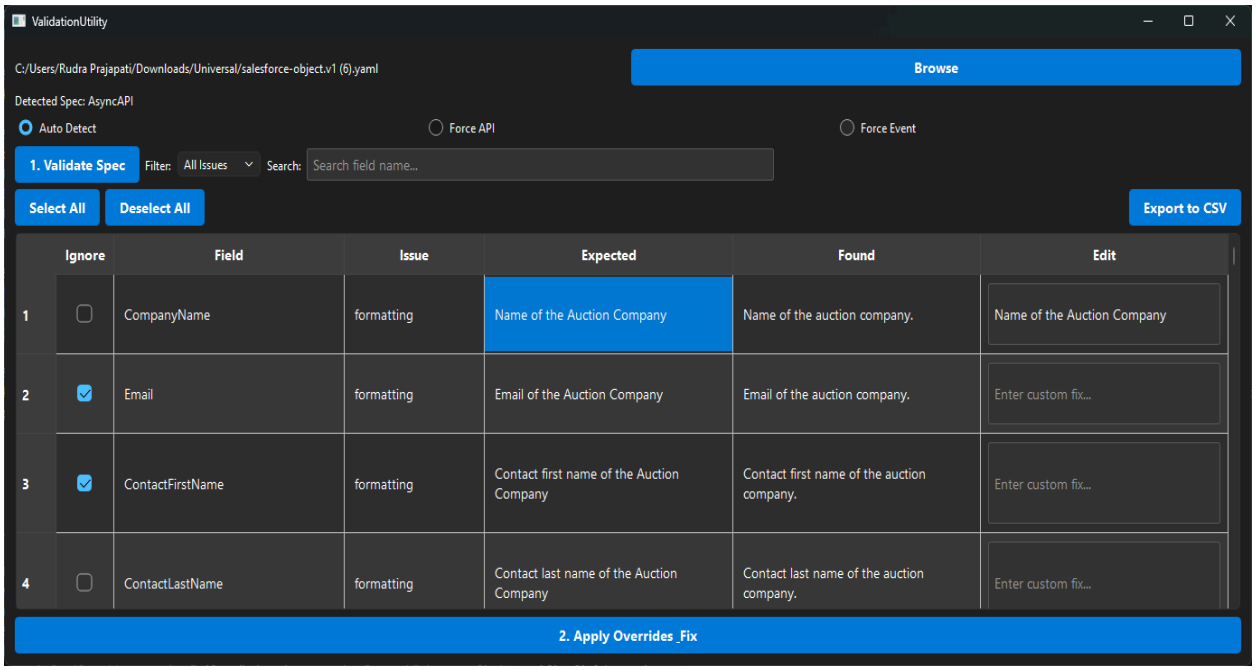
Found issues will appear in the central table. Each row represents a specific violation showing:

- **Field:** The name of the property.
- **Issue:** The type of violation (e.g., `datatype`, `formatting`).
- **Expected:** What the value should be.
- **Found:** The current incorrect value.



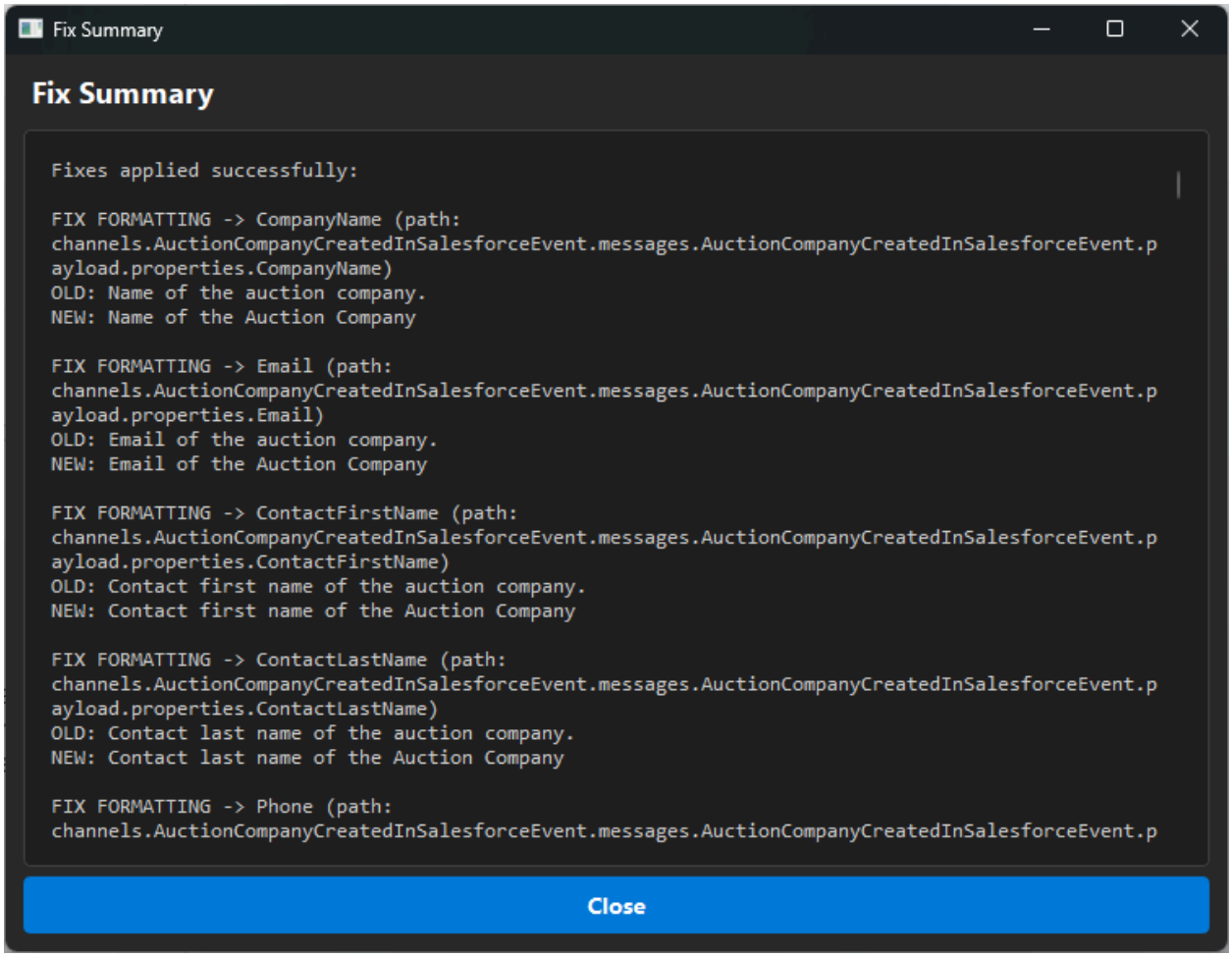
5. Apply and Ignore cases

- **Ignore:** Check the box in the "Ignore" column to skip a specific violation.
- **Edit:** Click the "Edit" field to manually override a suggestion, or use the radio buttons (if available) to pick from multiple suggestions.
- **Double-Click:** Quickly copy the "Expected" value to the "Edit" field by double-clicking the Expected cell.



6. Save Results

Click '2. Apply Overrides & Fix'. ValidationUtility will generate a new file named `[original_name]_updated.yaml` in the same directory as your source file. You can also click 'Export to CSV' to save a report of the violations.



Rules and Validation Logic

ValidationUtility uses a "Rule Engine" that traverses your spec. Here is how it identifies issues:

Naming Conventions

- API (OpenAPI): Enforces `snake_case` (e.g., `user_id`, `created_at`).
- Event (AsyncAPI): Enforces `PascalCase` (e.g., `UserId`, `CreatedAt`).

Data Type Validations

- Dates: Fields ending in `_at` (API) or `At` (Event) must be `string` with `format: date-time`.
- Booleans: Ensures boolean fields follow specific prefix/suffix rules:
- API: Starts with `is_` or ends with `_enabled`.
- Event: Starts with `Is` or ends with `Enabled`.

Description Logic

- Missing Descriptions: Automatically generates descriptions based on the field name (e.g., `is_active` -> "True, if active").
- Formatting: Removes common placeholder text like "(e.g., ...)", enforces sentence casing, and ensures proper terminal punctuation (full stops).
- Acronyms: Automatically capitalizes known acronyms like `ID`, `ISO`, `UTC`, `URL`, and `JSON`.

Error Handling

Common Issues & Solutions:

1. **"Detected Spec: Unknown"**

- Cause: The YAML doesn't contain standard OpenAPI/AsyncAPI tags.
- Solution: Use the 'Force API' or 'Force Event' radio buttons to override detection.

2. **"Invalid YAML" Error**

- Cause: Syntax error in your source file (e.g., indentation).
- Solution: Check your file in a YAML linter before uploading.

3. **File Not Saving**

- Cause: The directory is read-only or the file is open in another program.
- Solution: Ensure you have write permissions to the folder.

User Actions and Controls

Action	Description
Browse	Select your source YAML file.
Validate Spec	Trigger the rule engine to scan the file.
Ignore (Checkbox)	Tell ValidationUtility NOT TO change this specific line
Edit (Text Input)	Provide a custom value if the "Expected" suggestion isn't quite right.
Export to CSV	Download a spreadsheet of all found issues for reporting.
Apply Overrides	Execute the changes and save the new YAML file.

FAQs

Q: Does ValidationUtility support JSON files?
A: Currently, ValidationUtility is optimized for YAML. Converting JSON to YAML first is recommended.

Q: Where is my updated file?
A: It is saved in the same folder as your original file, with `_updated` appended to the name.

Q: Can I customize the rules?
A: Rules are currently hardcoded in `api_governance.py` and `event_governance.py` to ensure organizational standards.

Q: Does it support Backstage YAMLs?
A: Yes! ValidationUtility automatically unwraps the `definition` block inside Backstage `API` entities and re-wraps it after fixing.

Q: Why are some suggestions separated by `||`?
A: This happens when there are multiple valid ways to fix a field (e.g., `is_active` vs `active_enabled`). You can select your preference using the radio buttons in the table.

Note

Always verify the `_updated.yaml` file in a spec viewer (like Swagger Editor) to ensure the changes align with your expected API behavior.